BAB 2. TCP/IP Model

2.1 Tujuan

- Mahasiswa mampu melakukan identifikasi transmisi data menggunakan model TCP/IP
- Mahasiswa mampu melakukan identifikasi layer dari model TCP/IP
- Mahasiswa mampu menggunakan aplikasi untuk mengidentifikasi data dari model TCP/IP

2.2 Dasar Teori

2.2.1 OSI Layer

OSI adalah referensi komunikasi dari Open System Interconnection. OSI model digunakan sebagai referensi untuk membahas spesifikasi protokol. OSI model terdiri dari 7 layer seperti pada tabel 1.

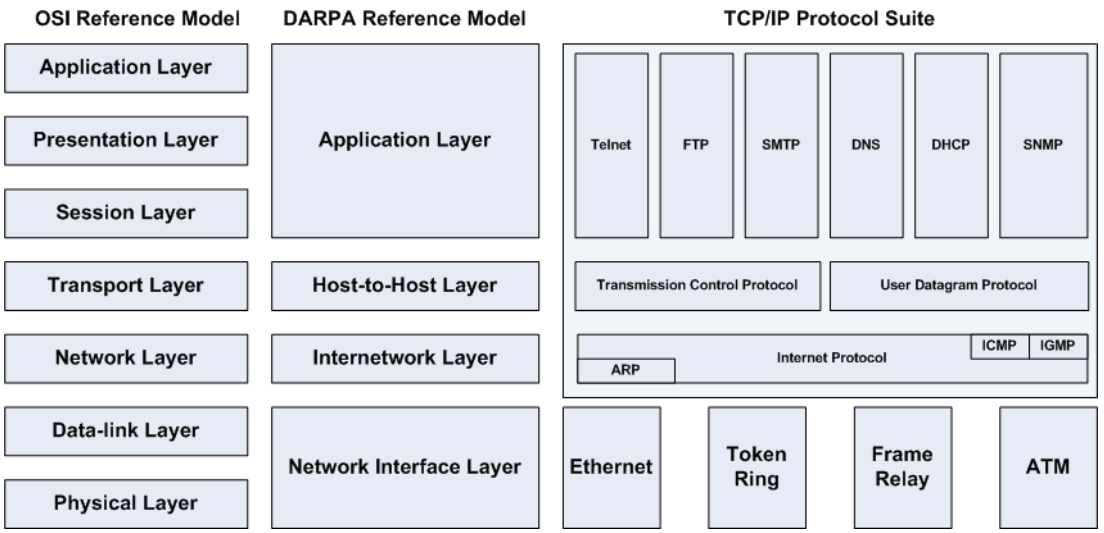
Layer #	Nama	Unit Enkapsulasi	Deskripsi/Fungsi
7	Application	Data	Mendefinisikan pelayanan (services) komunikasi pada suatu aplikasi
6	Presentation	Data	Mendefinisikan format data
5	Session	Data	Mendefinisikan memulai, pengontrolan dan mengakhiri komunikasi
4	Transport	Segment	Mendefinisikan jenis pengiriman
3	Network	Paket	Mendefinisikan pengiriman dari titik ke titik.
2	Data Link	Frame	Mendefinisikan cara pengaturan pengiriman data
1	Physical	bit	Mendifinisikan bentuk interface dari sebuah media transmisi

Tabel 1. OSI Layer

Dimana setiap layer memiliki fungsi dan contoh masing-masing.

2.2.2 TCP/IP Model

TCP/IP (singkatan dari Transmission Control Protocol/Internet Protocol), adalah gabungan dari protokol TCP dan IP sebagai sekelompok protokol yang mengatur komunikasi data dalam proses tukar-menukar data dari satu komputer ke komputer lain di dalam jaringan Internet yang akan memastikan pengiriman data sampai ke alamat yang dituju. Protokol ini tidaklah dapat berdiri sendiri, karena memang protokol ini berupa kumpulan protokol (Protocol Suite).
Arsitektur TCP/IP tidaklah berbasis OSI Layer, tetapi menggunakan model referensi DARPA. Seperti pada gambar 11., TCP/IP mengimplementasikan arsitektur berlapis yang terdiri atas empat lapis. Empat lapis ini, dapat dipetakan terhadap model OSI layer. Empat lapis ini, disebut juga DARPA Model atau DOD Model.



Gambar 11. OSI Layer vs TCP/IP Protocol Suite.

2.3 Pengenalan Aplikasi

2.3.1 Wireshark

Wireshark adalah aplikasi penganalisa paket jaringan (Network Packet Analyzer). Sebuah software penganalisa paket jaringan yang bekerja dengan cara menangkap paket jaringan dan mencoba untuk menampilkan data paket yang tertangkap dengan detail.

Wireshark bersifat open source.

Contoh penggunaan wireshark:

- Administrator jaringan menggunakan untuk melakukan penangan masalah jaringan
- Network security engineer menggunakan untuk memeriksa keamanan jaringan
- Developer menggunakan untuk melakukan debug pada implementasi protokol
- Umum digunakan untuk mempelajari internal protokol jaringan

Wireshark memiliki fitur:

- Tersedia untuk UNIX dan Windows.
- Menangkap paket data langsung dari network interface.
- Tampilan paket dengan informasi protokol yang sangat rinci.
- Open dan Save data paket yang dicapture.
- Import dan Eksport paket data dari dank e banyak program capture lainnya.
- Menyaring paket data dengan berbagai macam kriteria.
- Warnai layar paket berdasarkan filter.

2.3.2 Cara penggunaan Wireshark

Langkah-langkah menggunakan wireshark

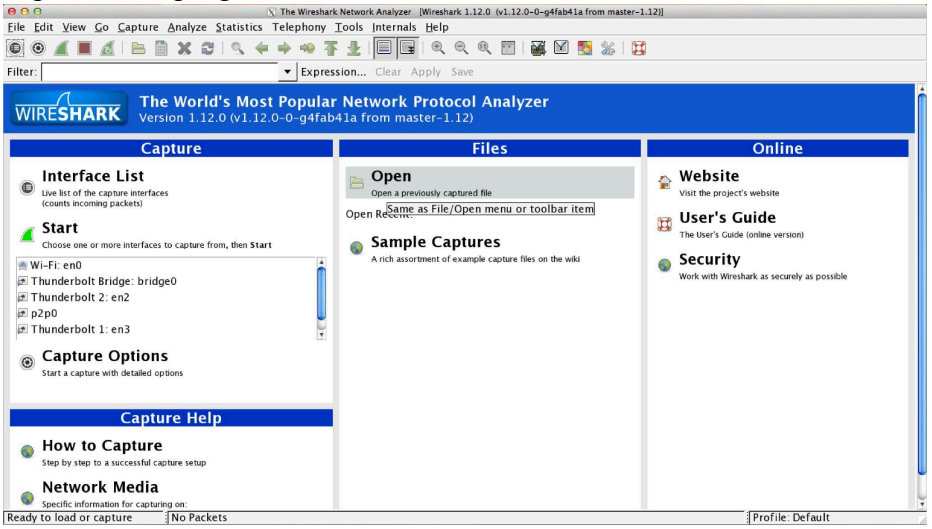
1. Klik ikon…



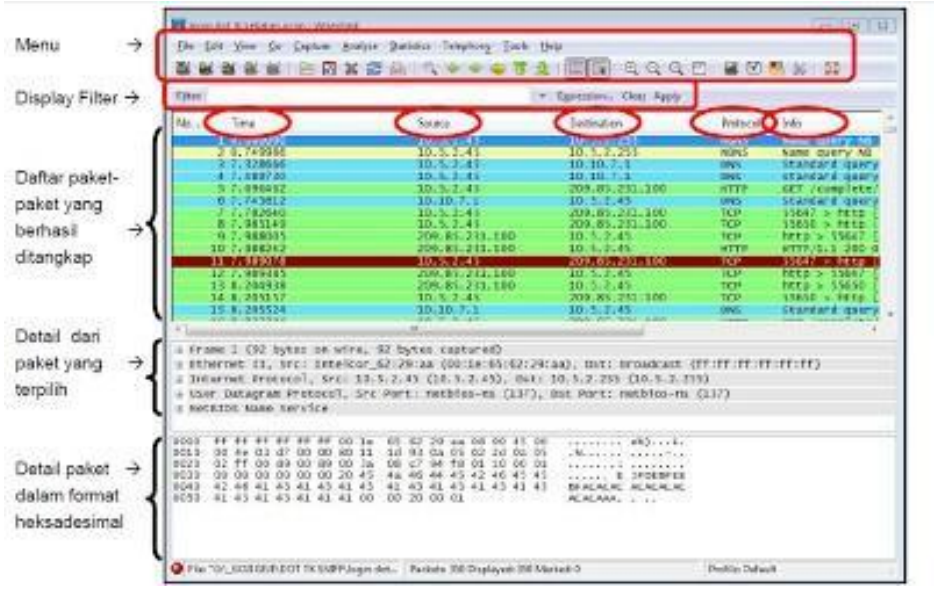
2. Kemudian akan tampil aplikasi wireshark dengan diawali spash screen.



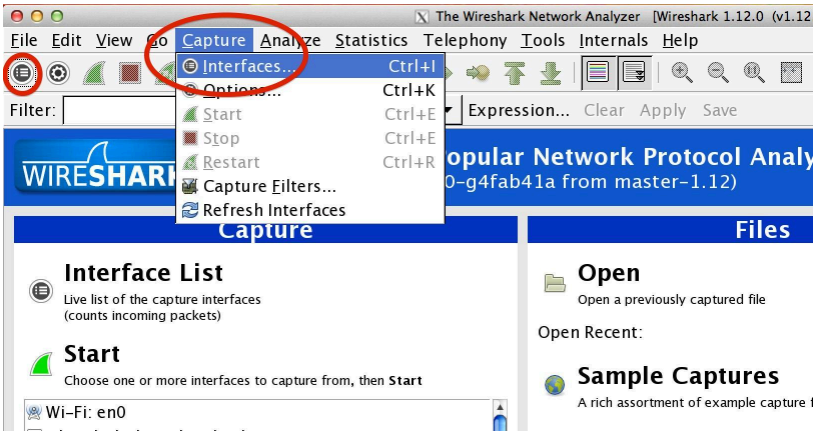
3. Tampilan awal program Wireshark.



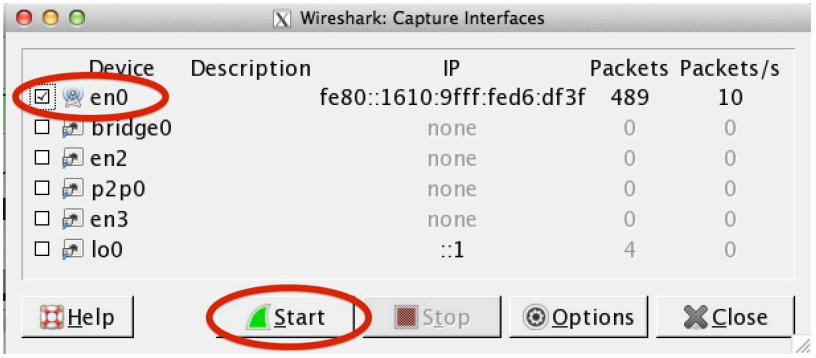
4. Keterangan menu wireshark.



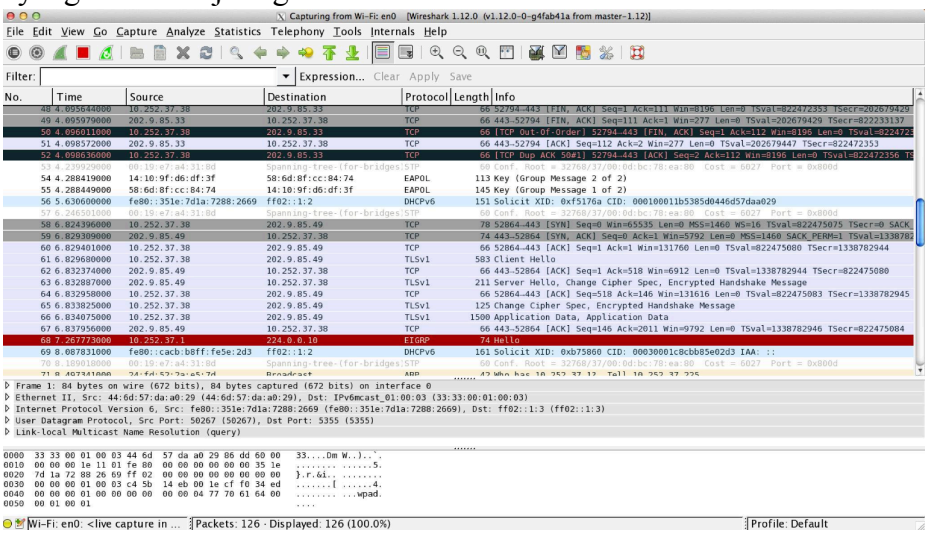
5. Melakukan CAPTURE data menggunakan wireshark. Pilih menu INTERFACE.



6. Pilih interface yang kita gunakan, kemudian klik start.



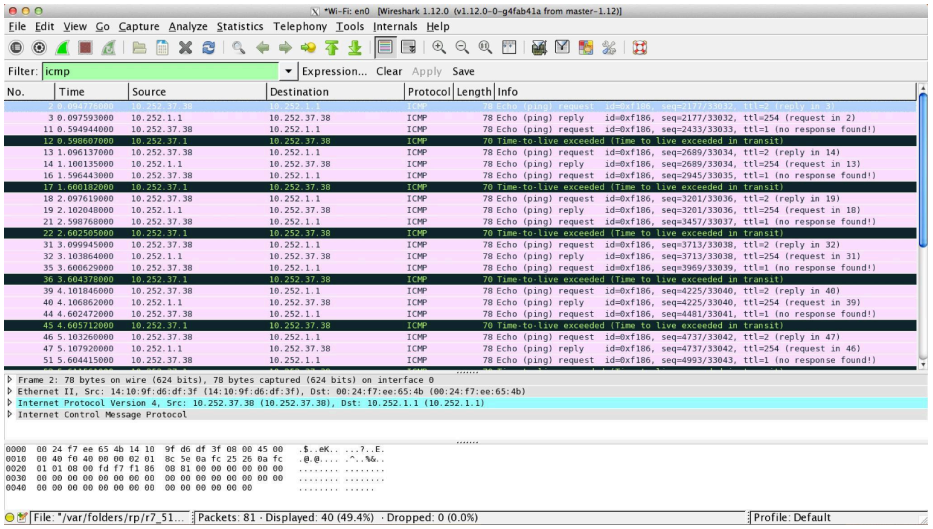
7. Setelah mengaktifkan interfaces, Wireshark akan beraksi membaca semua data yang berada di jaringan.



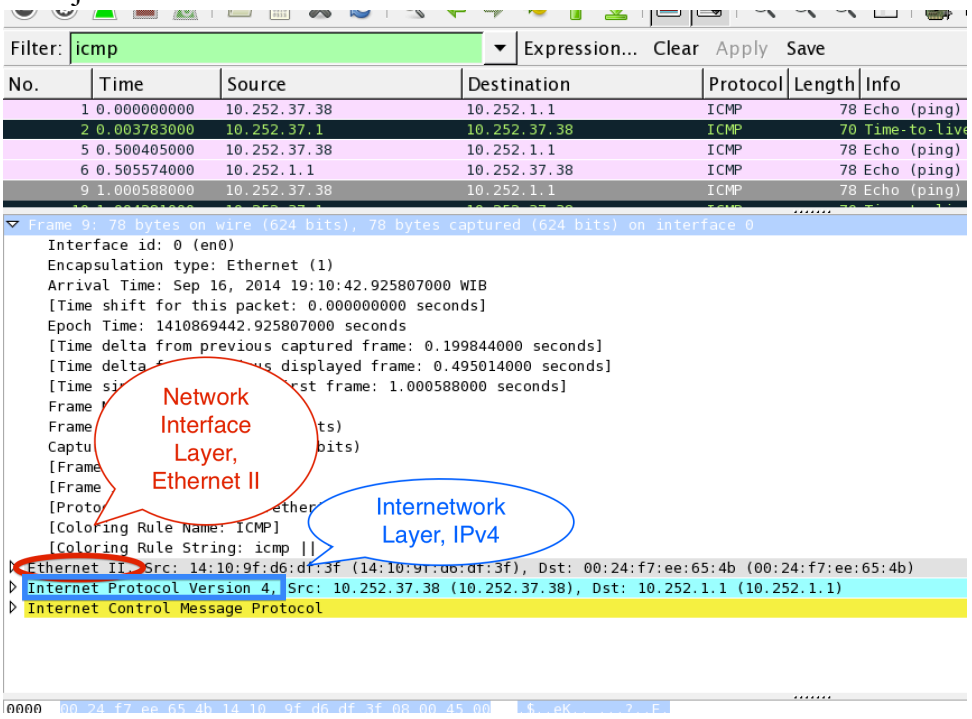
Terlihat banyak data yang dapat dicapture.
Keterangan:

Source	Alamat IP pengirim
Destination	Alamat IP tujuan
Protocol	Jenis protokol yang digunakan, seperti TCP, DHCP, dll
Length	Panjang data yang dikirimkan
Info	Informasi isi data yang dikirimkan

8. Ketikkan ICMP pada filter box untuk melakukan penyaringan data yang akan ditampilkan sehingga hanya data PING yang akan ditampilkan.



9. Penjelasan data



2.4 Persiapan Praktikum

1. Modul praktikum
2. PC yang dilengkapi dengan aplikasi Wireshark
3. Jaringan aktif
4. Lembar Laporan Praktikum (Lampiran)
5. Koneksi LAN dan Internet

2.5 Langkah-langkah Praktikum

2.5.1 Menjalankan aplikasi wireshark

1. Mahasiswa melakukan pengecekan apakah pada PC yang dipakai sudah terinstall aplikasi wireshark ?!!

2. Apabila belum terinstall, mahasiswa harus melakukan installasi di PC yang digunakan !!!

   Untuk OS Linux Debian/Ubuntu:
   dhoto@mypc $ su
   root@mypc # apt-get install wireshark

   Untuk OSX:
   - Download wireshark dan XQuartz
   - Lakukan konfigurasi XQuartz pada wireshark

   Untuk Windows:
   - Download wireshark
   - Klik 2x pada installer wireshark

3. Klik ikon Wireshark !!!
4. Klik pada menu CAPTURE dan memilih INTERFACES.
5. Pilih interface yang akan digunakan untuk mengrekam data.
6. Klik Start.
7. Pada OS linux, jalankan aplikasi TERMINAL dan ketik perintah PING yang digunakan untuk melakukan pengecekan jaringan (tanpa tanda $):
   $ ping 10.252.1.1
8. Kembali ke aplikasi wireshark, hentikan rekam data dengan cara klik tombol merah pada wireshark. Ambil data untuk jenis tipe: icmp dengan cara memfilter di aplikasi wireshark dan lakukan screen shot pada tampilan wireshark yang kalian gunakan.

2.5.2 Melakukan rekam data untuk jenis protokol tertentu

1. Lakukan praktikum seperti kegiatan 2.5.1 dari nomer 4 s/d 8.
2. Ganti kegiatan pada no 7, dengan kegiatan berikut:
   a. Jalankan aplikasi terminal dan ketik perintah NSLOOKUP yang digunakan untuk mengetahui IP dari suatu tujuan (tanpa tanda $):
      $ nslookup www.pens.ac.id
      Ambil data untuk jenis tipe: dns
   b. Jalankan aplikasi web browser dan arahkan ke alamat: ftp://newfs.pens.ac.id
      Download salah satu data yang ada di alamat tersebut !!!
      Ambil data untuk jenis tipe: ftp
   c. Jalankan aplikasi web browser dan arahkan ke alamat: http://www.pens.ac.id
      Ambil data untuk jenis tipe: http
3. Untuk masing-masing kegiatan diatas, laporkan data yang telah direkam sesuai dengan layer pada TCP/IP model. Gunakan lembar **Laporan praktikum 2. TCP/IP Model**. (Lampiran)

2.6 Lampiran

Laporan praktikum 2. TCP/IP Model

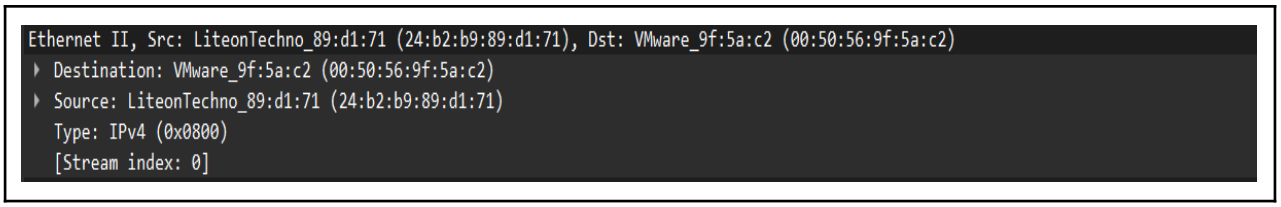
Kegiatan : rekam data untuk jenis protokol tertentu
Nama : Muhammad Niko Adithya Pratama
NRP : 5325600004

Data layer DNS

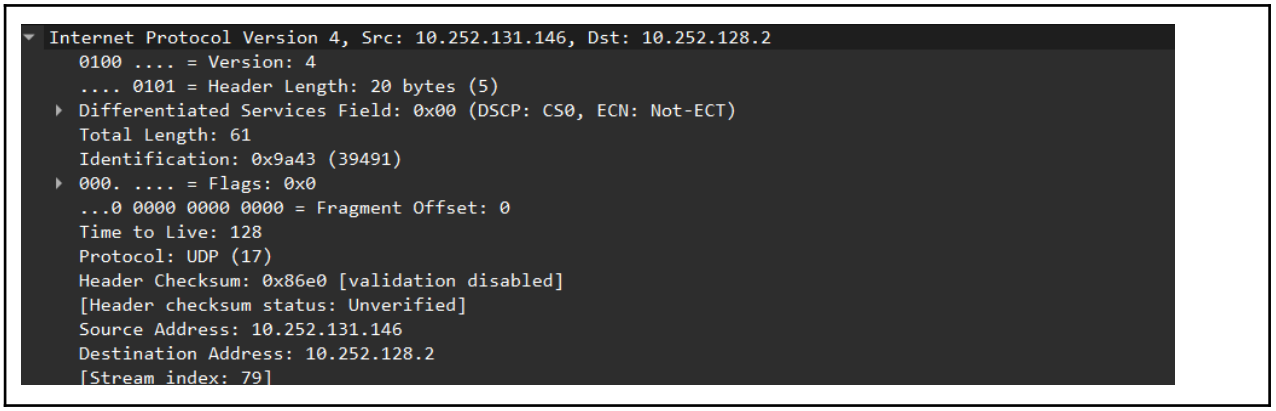
Application layer



Host-to-host layer



Internetwork layer

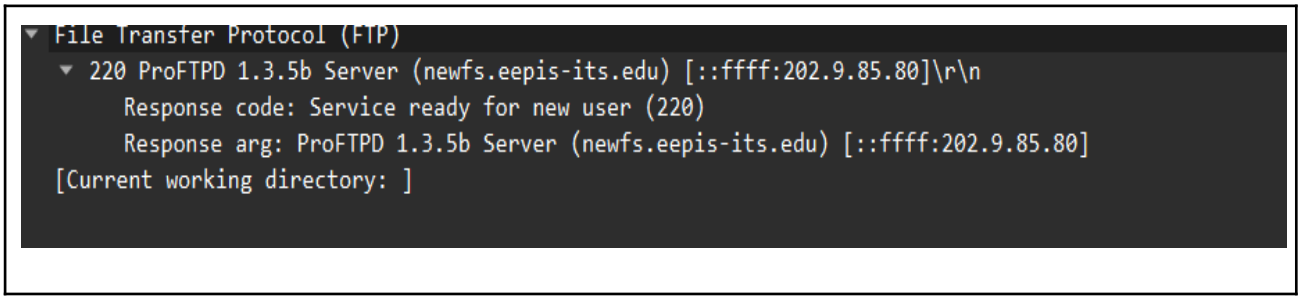


Network Interface layer

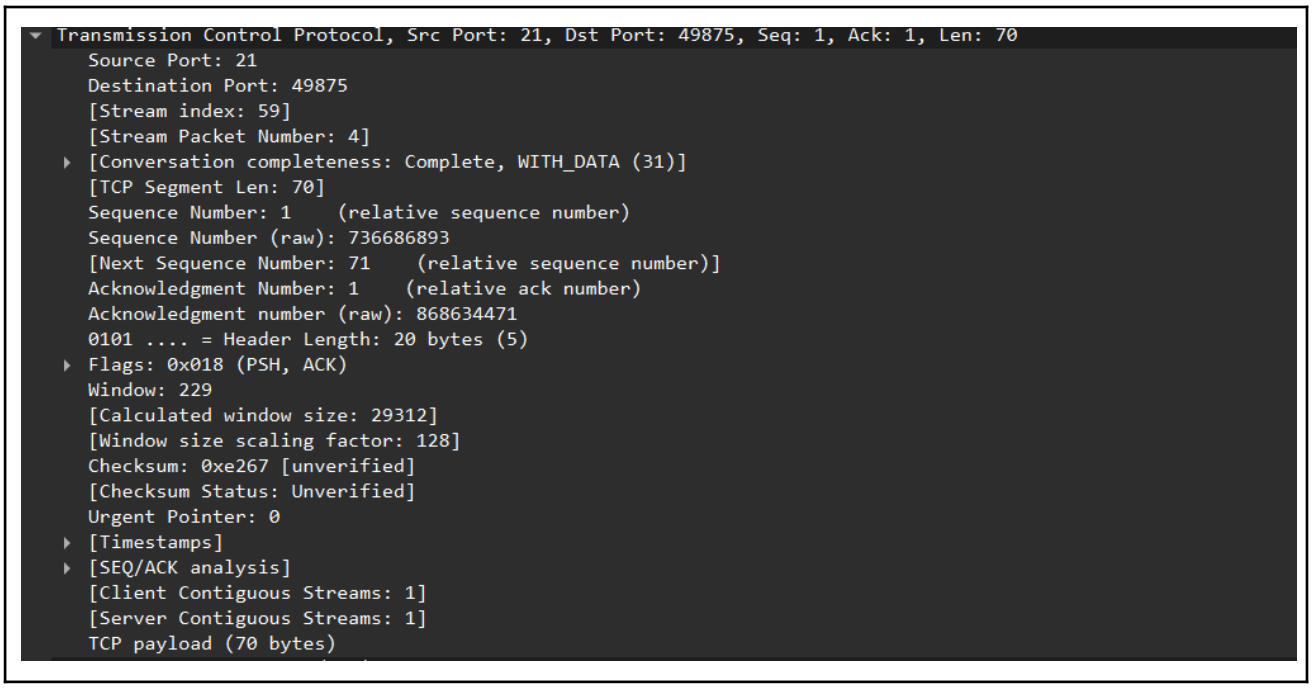


Data layer FTP

Application layer



Host-to-host layer



Internetwork layer

```
Internet Protocol Version 4, Src: 202.9.85.80, Dst: 10.252.135.22
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 110
    Identification: 0xec55 (60501)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: TCP (6)
    Header Checksum: 0x9dc8 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 202.9.85.80
    Destination Address: 10.252.135.22
```

Network Interface layer

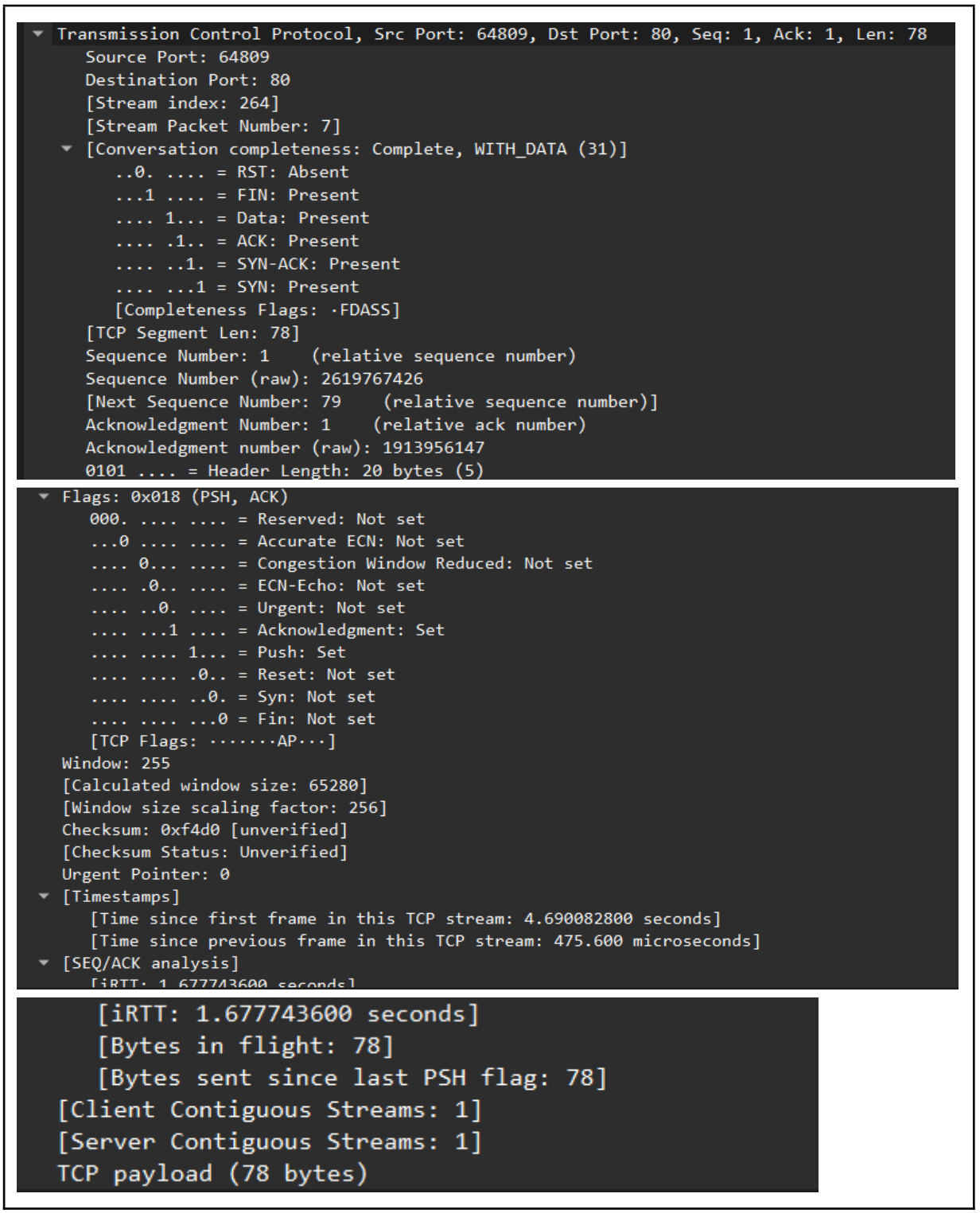
```
Frame 33313: Packet, 124 bytes on wire (992 bits), 124 bytes captured (992 bits) on interface \Device\NPF_{85EBA4C1-20A9-496C-BFD5-16C9BF04B313}, id 0
    Section number: 1
  ▸ Interface id: 0 (\Device\NPF_{85EBA4C1-20A9-496C-BFD5-16C9BF04B313})
    Encapsulation type: Ethernet (1)
    Arrival Time: Oct 13, 2026 12:01:07.332149500 SE Asia Standard Time
    UTC Arrival Time: Oct 13, 2026 05:01:07.332149500 UTC
    Epoch Arrival Time: 1791867667.332149500
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 7.523700 milliseconds]
    [Time since reference or first frame: 1 minute, 50.677884300 seconds]
    Frame Number: 33313
    Frame Length: 124 bytes (992 bits)
    Capture Length: 124 bytes (992 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp:ftp]
    Character encoding: ASCII (0)
    [Coloring Rule Name: TCP]
    [Coloring Rule String: tcp]
```

Data layer Http

Application layer

```
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
      Request Method: GET
      Request URI: /
      Request Version: HTTP/1.1
    Host: www.pens.ac.id\r\n
    User-Agent: curl/8.21.0\r\n
    Accept: */*\r\n
    \r\n
    [Response in frame: 335172]
    [Full request URI: http://www.pens.ac.id/]
```

Host-to-host layer



Internetwork layer

```
▾ Internet Protocol Version 4, Src: 10.252.135.22, Dst: 202.9.85.176
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▾ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      0000 00.. = Differentiated Services Codepoint: Default (0)
      .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 118
    Identification: 0x4410 (17424)
  ▾ 010. .... = Flags: 0x2, Don't fragment
      0... .... = Reserved bit: Not set
      .1.. .... = Don't fragment: Set
      ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x04a6 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.252.135.22
    Destination Address: 202.9.85.176
    [Stream index: 2365]
```

Network Interface layer

```
▾ Frame 334988: Packet, 132 bytes on wire (1056 bits), 132 bytes captured (1056 bits) on interface \Device\NPF_{85EBA4C1-20A9-496C-BFD5-16C9BF04B313}, id 0
    Section number: 1
  ▾ Interface id: 0 (\Device\NPF_{85EBA4C1-20A9-496C-BFD5-16C9BF04B313})
      Interface name: \Device\NPF_{85EBA4C1-20A9-496C-BFD5-16C9BF04B313}
      Interface description: Wi-Fi
    Encapsulation type: Ethernet (1)
    Arrival Time: Oct 13, 2026 12:22:25.503788500 SE Asia Standard Time
    UTC Arrival Time: Oct 13, 2026 05:22:25.503788500 UTC
    Epoch Arrival Time: 1791868945.503788500
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 475.600 microseconds]
    [Time since reference or first frame: 23 minutes, 8.849523300 seconds]
    Frame Number: 334988
    Frame Length: 132 bytes (1056 bits)
    Capture Length: 132 bytes (1056 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp:http]
    Character encoding: ASCII (0)
    [Coloring Rule Name: HTTP]
    [Coloring Rule String: http || tcp.port == 80 || http2]
```